

OPERATION IRON CHOIR: Command and Control, Its Containment, and a Botnet on an RP2350 Factory Floor Andon Station

Kevin Thomas
George Mason University
Fairfax, VA, USA
kthoma60@gmu.edu

Abstract: A station that never lies can still be a bot. The OPERATION IRON CHOIR build is a bare-metal RP2350 factory floor andon station and its companion factory control gateway, and it is Act VII of the OPERATION COLD IRON story. An andon node reads an SG90 servo as a line diverter, a VS1838B infrared receiver as a local fault-clear remote for FAULT CLEAR, ACK, and TEST, a DHT11 as the line temperature sensor, a 1602 LCD as the andon state, zone, temperature, and infection readout, red/yellow/green LEDs as the FAULT, ACK PENDING, and LINE OK tower light, a debounced button as the fault-clear request, and an RYLR998 LoRa link to a sealed factory gateway. Every request and command is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Act VII takes the exfiltration lesson of Act VI and turns it to obedience: a benign FROSTLINE command-and-control bot, compiled only under a SANDBOX_ONLY guard, emits a six-byte check-in frame whose preamble is C2V1 and whose body carries the bot identifier 0xB7, every four ticks; accepts a small set of benign remotely issued tasks (a tower light blink, a synthetic log line, and a status report); and erases and programs a real 0xC7 bot marker into the reserved flash sector 0x103FF000 with the Pico SDK flash API, so it re-installs on every later boot. It reads CoreDebug DHCSR at 0xE000EDF0 to suppress itself while a debug probe is attached. We document the peripheral set, the wire and envelope formats, the sealed fault path with its guarded command set, bounded zone band, anti-replay window, and authenticated state tag, the command-and-control bot and its task handler, the blue-half controls (sealed command path, fault-clear authorization, check-in and tasking gates, sector erasure, fail-safe diversion, build integrity), and an honest threat model that names the benign tasks, the local-script listener, the no-external- network scope, the shared lab key, the open debug port, and the deliberately inert bot as explicit decisions rather than accidents. A 146-case, 479-check native suite reaches 100% line coverage of every owned firmware module.

Index Terms: RP2350, factory andon, command and control, botnet, check-in, task handler, reserved flash sector, bot marker, XChaCha20-Poly1305, Argon2id, anti-replay, authenticated state, malware analysis, raw-frame listener, anti-debug, CoreDebug DHCSR, fail-safe diversion, embedded firmware.

I. Introduction

Act I of the OPERATION COLD IRON story was the silent lie: a cold-chain monitor that reported minus eighteen degrees while the store warmed. Act II was the door: an access gate that kept its final verdict in plain SRAM while the cryptography around it was correct.

Act III was the payload that is already inside: a valve controller carrying a benign implant that beacons, arms a logic bomb, and hides from a probe. Act IV was the payload that refuses to die: an HVAC node whose implant kept a copy of itself in a reserved flash sector and re-installed on every boot. Act V was the payload that spreads: a tamper ring whose worm handed its frame to

every neighbor. Act VI was the payload that steals: a drop-box that harvested a manifest and leaked it in the timing of legitimate frames. Act VII is the payload that takes orders. The dock led to the floor, and the floor led to the station. This device does not survive its removal and does not replicate across the mesh; it registers, it waits, and it obeys.

The factory floor andon station is that next device, and it is the floor’s own voice. Each node watches a line, reads the line temperature, verifies an authorized fault or clear command, drives a diverter, and reports its own health on an LCD with total confidence. The image pulled from the dock carried a manifest, and the manifest named a line. The thing that takes orders is already running in the station, and it has learned to answer a listener that the design never named. NorthPharma is the Ministry’s front; FROSTLINE wrote the leash.

The reveal that ties the seven acts together is the shift in what failure means. Act I was a lie about a number. Act II was a lie about a person. Act III was a lie about machinery, because a second party shared the chip. Act IV was a lie about the recovery, because the payload kept state the image did not own. Act V was a lie about containment, because the payload kept a copy outside the board. Act VI was a lie about confidentiality, because the payload gave the asset away while every light stayed green. Act VII is a lie about obedience, because the payload answers to a party the operator never authorized. The seven-act arc is a widening of the trust boundary: first the sensor, then the state, then the firmware image, then the removal procedure, then the network, then the wire, and now the chain of command.

An andon node is a simple machine. A temperature sensor reports the line, a gateway authorizes a command, an actuator moves a diverter, and a tower light tells the floor whether a person is needed. Three properties must hold at once: integrity, so the command that reaches the diverter is the authorized one; authority, so a local input cannot bypass the decision; and state, so the controller does not trust a stale or tampered verdict. The naive controller collapses all three. Act VII both fixes that and then goes further, because the command-and-control act has to answer a question a protocol cannot: what does it mean when the traffic is well formed, the cryptography is correct, and the device is still taking orders from somewhere else?

The classroom goal is to teach both halves. The red half and the malware track find the defects: forge a command, replay a captured command, read the reserved sector, stop the task handler, clear the bot marker, inject the magic on the raw frame, and step past the anti-debug trap. The blue half and the fix track seal the station: a sealed and guarded fault path, a monotonic anti-replay window, an authenticated state tag, a fault-clear request that asks for authorization instead of bypassing it, a fail-safe divert policy, closed check-in and tasking gates, sector erasure, and build-level integrity. The centerpiece is a lesson about control: the wire is authenticated, the verdict is tagged, and the station still obeys, because the bot never needed the wire and it answers a listener the protocol never authenticated.

I.I. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 factory floor andon station that drives the full Embedded Hacking peripheral set: an SG90 line diverter actuator, a VS1838B NEC fault-clear remote with FAULT CLEAR, ACK, and TEST commands, a DHT11 line temperature sensor, a 1602 LCD andon readout over I2C, a red/yellow/green tower light, a debounced fault-clear request, and RYLR998 command traffic with a declared-length payload parser.
- A sealed fault command path with a guarded command set, a bounded zone band, a monotonic anti-replay sequence window, and an authenticated state tag that detects a debugger-written verdict before the diverter moves.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the andon node identifier bound as associated data.
- A benign FROSTLINE command-and-control bot, confined to a SANDBOX_ONLY build, that demonstrates a raw-frame magic listener, a six-byte C2V1 check-in with the 0xB7 bot identifier, a benign task handler, a real reserved-sector bot marker, re-install on boot, and a CoreDebug DHCSR anti-debug trap.
- A factory control gateway that authenticates before it parses, logs authenticated and rejected requests

distinctly, and answers only authenticated requests with a sealed command, plus a spoofing client whose forged and replayed commands are rejected.

- A corpus-aligned packet artifact contract (`packet_artifact.json` / `packet_artifact.h`) with a build-time staleness guardrail.
- A 146-case, 479-check native test suite reaching 100% line coverage of every owned firmware module, including the bot, the check-in, the task handler, the gates, and the marker paths, checked against published RFC test vectors.
- A threat model that states explicitly what the lab profile does and does not protect, and an honest account of the bot as a sanitized educational artifact whose listener is a local script, whose tasks are benign, and whose scope has no external network.

II. Related Work

Industrial and factory monitoring is a mature field, and the andon station is a canonical target. The DHT11 one-wire sensor [1] and the NEC infrared remote encoding [2] are broadly documented and representative of the temperature and operator surfaces real installations deploy. The authenticated construction we use follows the ChaCha20-Poly1305 standard [6], and Argon2 follows the Argon2 specification [7]. The stateful construction is the standard replay defense found in secure-messaging and payment protocols, applied here at the scale of one diverter.

Three lines of work frame Act VII. The first is the long line of firmware implants and logic bombs [8]: code that lives on the device, waits for a trigger, and acts through the device’s own actuators rather than through its protocol. The second is command and control and botnets: a payload that registers with a listener, checks in on a cadence, and executes remotely issued tasks, so an operator can drive a fleet in concert. The third is persistence and re-install: a payload that keeps a copy of its state outside the firmware image so a reflash does not remove it. The C2V1 preamble, the 0xB7 bot identifier, the benign task handler, the reserved-sector marker, and the CoreDebug DHCSR check used here are minimal, well-known examples of those techniques, chosen because they are legible on a debug probe and cheap to verify.

The pedagogical use of intentionally vulnerable firmware is established [5]. The difference in this act is that the artifact does not attack the integrity of the protocol

and does not attack the confidentiality of a payload; it attacks the control of the device while leaving every observable health signal intact. The exercise demonstrates the check-in, the tasking, the marker, the trap, and the complete removal on the same board, and it makes the scope limit explicit: an authenticated link does not authenticate the listener, a sealed command path does not stop a handler that reads the raw bytes underneath it, and a green lamp does not mean the station is taking orders from the factory.

III. System Model

The system consists of four roles:

- **Andon station node (RP2350 firmware):** decodes the infrared fault-clear remote, verifies and authorizes sealed gateway fault and clear commands, annunciates the tower light, checks the DHT11 line band, drives the servo diverter, handles the fault-clear request, renders the andon readout, and (SANDBOX_ONLY) runs the command-and-control bot.
- **Factory control gateway (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to `andon_log.csv`, decides authorization, and answers an authenticated request with a sealed command carrying a monotonic sequence number and an authenticated state tag.
- **Edge simulator:** a laptop process that behaves like an additional station, sealing zone requests with the same field key.
- **Attacker:** a laptop process that claims the gateway address, forges a command, or replays a captured command at the node.

Let

$$A \in \{0, 1\}^{\{16\}}$$

be the LoRa node address,

$$L$$

the declared payload byte length, and

$$C$$

the ASCII payload, which is a lowercase hex envelope. The unauthenticated wire framing is:

$$+RCV=, A, ,, L, ,, C, ,, rssi, ,, snr, CRLF$$

Because the hex payload has no commas, the framing is simpler than Act I's comma-bearing JSON, but the receiver still slices by declared length rather than by counting delimiters, for exactly the reason Act I documents.

III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is identical to Acts I to VI so one breadboard serves all seven, and it is fixed in `include/andon.h` and enforced by the native test suite:

Signal	RP2350 GPIO
DHT11 line temperature sensor (one-wire)	GP4
1602 LCD SDA (I2C1)	GP2
1602 LCD SCL (I2C1)	GP3
RYLR998 RX (UART1 TX)	GP8
RYLR998 TX (UART1 RX)	GP9
Infrared fault-clear remote (VS1838B)	GP5
Line diverter actuator (SG90 PWM)	GP14
Fault-clear button	GP15
Red FAULT LED	GP16
Yellow ACK PENDING LED	GP17
Green LINE OK LED	GP18
Onboard heartbeat LED	GP25

The LCD backpack uses the PCF8574 at 7-bit address `0x27`. The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the diverter moves; retracted is 0 degrees and deployed is 90 degrees. At boot the node programs its own transceiver (`AT+ADDRESS=7`, `AT+NETWORKID=18`) and the gateway programs the receiver (`AT+ADDRESS=1`, `AT+NETWORKID=18`) before logging, so command traffic is only delivered between radios that share the network identifier.

The line diverter is fail-safe: it is retracted at initialization and driven to the divert posture on every failure path, so loss of power, a failed temperature read, a malformed command, a tampered verdict, or a lost link all leave the gate diverted and the zone returned to the fail-safe value (0). The fault-clear request is a request,

not an authorization, and it never moves the gate on its own.

III.II. Operator Remote, Line Sensor, and Annunciation

The VS1838B is a 38 kHz demodulating infrared receiver whose output idles high and pulls low during a mark. The decoder times edges and reconstructs a NEC pulse train, then feeds the command into the request set. `ANDON_IR_FAULT_CLEAR` is `0x01`, `ANDON_IR_ACK` is `0x02`, and `ANDON_IR_TEST` is `0x03`. The optical surface has no key and no challenge, so a fault clear is treated as a request, not as an authorization; the sealed radio path is what moves the diverter in the defended design, and the optical path is a surface the red half examines.

The DHT11 is the line temperature sensor. A reading that fails its checksum is never safe, and a valid reading outside the band (`ANDON_TEMP_MIN_TENTHS = 0` to `ANDON_TEMP_MAX_TENTHS = 400`, that is 0.0 C to 40.0 C) is out of band. Either case marks the line as not nominal, so a dead or unplugged sensor, or a genuinely unsafe line, is visible in the andon readout.

Exactly one tower light lamp is lit at a time. Red is FAULT, yellow is ACK PENDING while a fault clear awaits authorization, and green is LINE OK. The 1602 LCD shows the andon state and the link on line one (`ST:OK L:UP`) and the zone and the infection status on line two (`Z:4 T:235 I:--`).

IV. Wire Protocol

The fault-clear control or the edge simulator seals a two-byte zone into an XChaCha20-Poly1305 envelope and sends it to the gateway:

```
AT+SEND=0001,84,<84 lowercase hex characters>
```

The gateway answers an authenticated request with a sealed andon command. The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2]
(little-endian) || tag[16]
```

where `seq` is the monotonic gateway sequence number, `command` is one of the guarded andon commands `ANDON_COMMAND_FAULT` (`0x01`), `ANDON_COMMAND_CLEAR`

(0x02), or `ANDON_COMMAND_ACK` (0x03), `zone` is the authorized line zone in the 0 to 16 band, and `tag` is a tag over the authorization record the command would produce. The gateway sends the reply back to the claimed sender:

```
AT+SEND=<node>,126,<126 lowercase hex characters>
```

The radio's AT command buffer (`RADIO_AT_CMD_MAX_LEN`), the inbound +RCV buffer (`RADIO_RCV_MAX_LEN`), and the generated artifact limit (`PACKET_MAX_RCV_LEN`) are all 256 bytes, which comfortably holds the largest possible envelope plus framing. The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

The firmware enforces a guarded command set and a bounded zone band in `control_parse`: the recovered command byte must be one of the three guarded andon codes, and the recovered zone must lie between `ANDON_ZONE_MIN` (0) and `ANDON_ZONE_MAX` (16). This is the sealed replacement for the unauthenticated fault injection, and it means a raw value or an out-of-band zone can never reach the diverter decision.

IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout:

```
nonce[24] || ciphertext[L] || tag[16]
```

For a two-byte request body this is $24 + 2 + 16 = 42$ bytes, or 84 hex characters. For a 23-byte command body this is $24 + 23 + 16 = 63$ bytes, or 126 hex characters. The maximum plaintext is 48 bytes (`ENVELOPE_MAX_PLAINTEXT`), so the largest possible envelope is $24 + 48 + 16 = 88$ bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (`ENVELOPE_MAX_HEX_LEN`). The declared length

L

in the framing is the length of the hex string, not of the underlying plaintext.

IV.II. Declared-Length Slicing Invariant

Given the substring

T

after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant

$$T[L] = ,$$

is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is the same discipline Act I adopts for comma-bearing JSON, retained here for uniformity and for defense against a hostile declared length.

V. Cryptographic Design

The radio is the first open path, and it is the one a key can close; the bot is the second open path, and it is one a key cannot close. The design goal is that a forged or modified frame must fail before any decision is made, while acknowledging that a payload which answers its own listener on the raw bytes of well-formed frames is unaffected by an integrity check. Two primitives provide the first property, and both are implemented in this repository with no third-party code.

V.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [7] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte key at initialization with the classroom profile $t=3, p=1, m=64$ blocks (`CRYPTO_KDF_TIME_COST`, `CRYPTO_KDF_PARALLELISM`, `CRYPTO_KDF_MEMORY_BLOCKS`). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes `coldiron-salt-01`. The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash H', and the RFC 9106 known-answer test runs in the Python suite.

V.II. XChaCha20-Poly1305 per Frame

Every frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [6] with an extended-nonce construction. The 24-byte

nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source, so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability

$$2^{-128}$$

The associated data is the andon node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node's frame.

V.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`. A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

V.IV. Key Model

Act VII uses a single field key. It seals every frame on the wire and it computes the state tag over the authorization record. In the classroom build the field key is derived from one committed lab passphrase and salt, so the firmware and the gateway interoperate with

no provisioning step. That is a lab convenience, not a deployment, and the documentation says so. The design keeps the roles separable so a student can reason about the real lifecycle: derive, provision per device, use, rotate on a schedule, and retire. A production build provisions key material from one-time-programmable (OTP) memory and keeps the state-tag key off the field device where possible. The bot is deliberately orthogonal: it is never given a key, it never opens an envelope, and it demonstrates that a valid key does not stop an adversary who never needs one.

VI. Anti-Replay Window and Authenticated State

Strong AEAD is necessary and not sufficient. Two stateful controls sit above the sealed wire.

VI.I. The Anti-Replay Window

A captured command is authentically sealed, so a controller that checks only the tag will happily apply it again. The authorization record keeps `last_seq`, the highest sequence number ever accepted, and `andon_auth_apply` accepts a command only when its sequence is strictly greater than `last_seq`. The order of checks is deliberate: the sequence test is evaluated first, then the tag is verified against the candidate record the command would produce, and only then is the record updated. A replayed valid command therefore fails on freshness, not on cryptography, which is exactly the lesson: authentication is not freshness.

VI.II. The Authenticated State Tag

The centerpiece is the verdict itself. The controller decides with a boolean in SRAM, call it `granted`, and an attacker with a debug probe and a GDB session does not break the cipher; they set `granted = true`. To detect that, the authorization record is nine bytes:

```
record = granted[1], seq[4], last_seq[4]
```

and the state tag is an XChaCha20-Poly1305 tag over that record, computed under the field key with a deterministic nonce built from the sequence number and the domain byte 0xA7:

```
tag = AEAD_seal(fieldkey, nonce(seq), record, AD = 0xA7)
```

`andon_auth_state_ok` recomputes the tag and compares it in constant time, and the guarded command path requires it before the diverter moves. A debugger that flips `granted` without recomputing the tag changes the record, so the stored tag no longer matches and the release is denied ahead of the actuator. The wire is authenticated, and so is the verdict.

VI.III. TOCTOU in One Session

The two attacks are independent and teach different defaults. The window stops a valid command from working twice. The tag stops an unauthorized verdict from existing at all. Together they convert the original failure, a correct decision followed by a mutable state read (a time-of-check to time-of-use gap), into two explicit, testable checks.

VII. Envelope Layout and Gateway Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

```
envelope = nonce[24], ciphertext[L], tag[16]
```

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time, and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the gateway side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the `+RCV` line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as `UNAUTHENTICATED` with an empty zone and stop. The forged body is never parsed.
3. Only for an authenticated frame, recover the two-byte zone, check it against the bounded band, write an OK row, and answer with a sealed command carrying the

next monotonic sequence and the state tag over the record the command would produce.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `zone`, `rsi_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope, and in replay mode it can only resend a captured command that the window will refuse.

VIII. The FROSTLINE Command-and-Control Bot: Check-In, Tasking, and Bot Marker

Act VII is the command-and-control act. The bot is real in technique and inert in effect, and it is confined to a single module compiled only under a build guard. This section states what it does, how it registers, how it hides, how it is detected, and the honest limit of the artifact.

VIII.I. Build Guard and Safety Boundary

`src/implant.c` is compiled only when `SANDBOX_ONLY` is defined. The clean firmware build does not define it, so the shipping image contains no bot. The native test build and the companion CTF build do define it, and the test build also defines `IMPLANT_HOST MOCK`, which replaces the CoreDebug register and the reserved flash sector with controllable host variables. This is the containment boundary: the malware track is a build configuration, not a hidden runtime feature of the shipping firmware. There is no network, no filesystem, and no host impact; the reserved sector is on the same chip and holds nothing else, and the only effect of the bot is on the student's own radio traffic and the infection field on the student's own LCD.

VIII.II. The Check-In and the Benign Task Handler

`implant_check_in` builds a six-byte frame: the 4-byte magic `C2V1` (`ANDON_IMPLANT_C2_MAGIC`), the bot identifier `0xB7` (`ANDON_IMPLANT_BOT_ID`), and a zero task byte, and sends it with `radio_send_frame` to the provisioned gateway address. `implant_tick` advances a

monotonic counter and, while the bot is armed, check-in is enabled, and no probe is attached, emits a check-in every `ANDON_IMPLANT_TICK_INTERVAL` (4) ticks. The bot announces itself on a fixed cadence whether or not a listener is present.

`implant_handle_command` matches the 4-byte C2V1 preamble on the raw inbound payload with `implant_magic_match`, which is a `memcmp`. On a match, and when tasking is enabled and no probe is attached, it arms the payload, writes the bot marker, records the task, and executes it. `implant_execute` runs `ANDON_IMPLANT_TASK_BLINK` (0x01, a tower light blink), `ANDON_IMPLANT_TASK_LOG` (0x02, a synthetic log line), or `ANDON_IMPLANT_TASK_REPORT` (0x03, a status report frame back to the listener), and ignores an unknown task code. Every task is benign and bounded.

VIII.III. Reserved-Sector Bot Marker and Re-Install on Boot

`implant_init` reads the marker byte at `ANDON_IMPLANT_RESERVE_ADDR` (0x103FF000), the final sector of external flash. On the first run the marker is absent, so the implant erases the sector and programs 0xC7 (`ANDON_IMPLANT_MARKER_BYTE`) with the real Pico SDK flash API, `flash_range_erase` and `flash_range_program`, exactly once. On every later boot the marker is present, so the bot re-arms its payload handler without any firmware change. A reflash that rewrites the program region does not touch the reserved sector, so the bot survives the procedure that was supposed to remove it. The marker is not a simulated memory-mapped store; on real silicon the erase and program calls are the only operations that persist it.

VIII.IV. The C2V1 Preamble and the Tasking Path

The bot preamble is the 4-byte string C2V1 (`ANDON_IMPLANT_C2_MAGIC`). One task frame is six bytes: the magic, the bot identifier, and a one-byte task code. `implant_handle_command` validates the frame against the tasking gate and the anti-debug check, and only then arms, writes the marker, and executes. Everything about the path is a `memcmp` and a small switch, which is the point: a command- and-control listener does not need a protocol, only a device that will answer.

`implant_tick` advances the monotonic tick counter. While the bot is armed and unobserved, every `ANDON_IMPLANT_TICK_INTERVAL` (4) ticks the bot checks in. A floor of these nodes, each calling the same listener on the same cadence, is the fleet that the Ministry can drive in concert. Nothing in this path opens an envelope or presents a key.

VIII.V. The Raw-Frame Magic Handler

The decisive detail is where the bot listens. `monitor_apply_frame` delivers the raw inbound payload to `monitor_implant_frame` (and thus to `implant_handle_command`) before it calls the sealed `control_handle_frame`. The bot therefore matches the 4-byte C2V1 preamble on the raw payload, underneath the envelope, and it never presents a sealed frame of its own. A forged, unauthenticated, or replay-protected command is irrelevant to a payload that reads the bytes before authentication. That is the scope lesson of the act: a correct sealed command path does not protect a listener that sits below it.

VIII.VI. The Anti-Debug Trap

Every tick, and every task frame, `implant_debug_attached` reads the CoreDebug DHCSR register at 0xE000EDF0 (`ANDON_IMPLANT_DHCSR_ADDR`). Bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT` (`ANDON_IMPLANT_DHCSR_DEBUGEN` and `ANDON_IMPLANT_DHCSR_HALT`). When either bit is set, `implant_ready` and `implant_tasking_frame` return false, so the check-in and the task handler are both suppressed while a probe is attached. The bot behaves benignly under observation and resumes when the probe is detached. This is the minimal anti-analysis trap, and it is deliberately simple so a student can see the branch, set a breakpoint after it, and prove the payload.

VIII.VII. Detection and Neutralization

The bot is detected by image comparison: the clean build and the `SANDBOX_ONLY` build differ by the implant module and its symbols. It is detected by the reserved-sector marker at 0x103FF000, which is state the firmware image does not own. It is detected on the link by the six-byte C2V1 frame, its 0xB7 bot identifier, and its four-tick cadence, and on the LCD by the `I:INF` infection field.

It is detected by static analysis by the C2V1 magic, the task codes, and the DHCSR read address. It is detected under GDB because the DHCSR read is a branch that a student can stand after. The native bot tests assert each behavior and its containment.

Neutralization is not a one-byte patch. It is the closure of the check-in gate, the closure of the tasking gate, the clearing of the runtime state and the bot marker, the erasure of the reserved sector, the removal of the code path, and the removal of the build flag, plus image signing and a debug lockdown on a deployed part. In the lab, the anti-debug trap is defeated by understanding the branch, not by hiding from it.

VIII. VIII. Honest Limitation

The bot is a benign educational command-and-control implant. It is confined to the breadboard, guarded by `SANDBOX_ONLY`, and has no network. It transmits only over the student's own LoRa modules on the classroom network id, it actuates only the tower light blink task, it executes only benign tasks, and it writes only to a reserved sector on the same chip that holds nothing else. The C2 is a local classroom script, run by the student over the student's own radio; there is no external address, no internet path, no remote server, and no external network of any kind. It is a demonstration of technique, not tradecraft: it does not load a second stage, its tasking is a bare memcmp, it does not survive a deliberate sector erase, and it does not resist a determined physical attacker. The value of the exercise is that it makes the control limit of an authenticated protocol concrete: a validated frame does not prove a validated listener, and a green station is not a station that answers only to the factory.

IX. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "iron-choir-andon-packets-v1",
  "frame_version": 1,
  "node_address": 7,
  "gateway_address_hex": "0001",
  "frame_size": 48,
  "link_wait_ms": 5000,
  "deploy_pulse_us": 1500,
  "retract_pulse_us": 500,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
```

```
"max_rcv_len": 256,
"example_frame": "{\\"evt\\":\\"fault\\",
\\"line\\":1042}"
}
```

`scripts/gen_packet.py` emits `include/packet_artifact.h` from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

X. Fail-Safe Policy and Fault-Clear Authorization

A diverter has a safe state, and the controller must choose it deliberately. The line is **fail-safe**: `diverter_init` retracts the gate at boot, `diverter_fail_safe` deploys it to the divert posture and records the fault state, and the monitor calls `monitor_fail_safe` on link loss, which also returns the fail-safe zone (0). A lost gateway link for longer than `ANDON_LINK_WAIT_MS` leaves the gate diverted and the safe zone active, because a command that cannot be authorized must not be assumed.

The fault-clear request is a local request, and it must not silently bypass authorization. `monitor_handle_clear` and `monitor_apply_ir_command` raise `g_clear_pending`; they never retract the diverter on their own. `monitor_apply_command` clears the pending indication only when an authorized command arrives. A one-button request therefore cannot outrank a sealed, authorized command. The lesson is that fail mode, priority, and the difference between a request and an authorization are policy choices, and naming them is part of the design.

XI. Attack Exercises and Hardening

The classroom runs the malware track and the fix track against the same build.

XI.I. Malware Track: Check-In, Tasking, Marker, and Trap

Students flash the `SANDBOX_ONLY` image, read the `0xC7` marker in the reserved sector `0x103FF000`, and prove they can erase it. They watch the link and locate the C2V1 check-in frame, its `0xB7` bot identifier, and its four-tick cadence, then close the tasking gate and prove an inbound task is ignored. They reflash the firmware, boot again, and, with a marker present, watch the bot re-arm. They then inject the C2V1 magic on the raw payload and observe the node arm, write the marker, and execute the benign task, with no sealed envelope and no field key. Finally they attach a probe, observe that the bot suppresses itself, break after the `DHCSR` check, and prove the payload with the trap bypassed. The centerpiece is the scope claim in one session: a correct, sealed command path does not protect a listener that reads the raw bytes underneath it, and a station whose health signals are valid can still be a station that takes orders.

XI.II. Red Half: Forgery and Replay

A structurally plausible command with a random nonce and a random tag is injected with `scripts/spoof.py --mode bad-tag`. The spoof tool holds no field key, so the tag cannot verify, and the controller denies before parsing any body. A captured command is replayed with `--mode replay`; the sequence is not greater than `last_seq`, the command fails on freshness, and the `FAULT` lamp lights. This is a pedagogical reintroduction of a well-known link failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

XI.III. Red Half: The Verdict in SRAM

The exercise halts the controller under the Debug Probe, breaks in the authorization path, sets `granted = true`, and continues. Under a controller that trusts the boolean the diverter moves. Under the Act VII controller the state tag is recomputed over the modified record, the mismatch is found, and the command is denied ahead of the actuator.

XI.IV. Blue Half: Sealing It

The blue-half controls map one-to-one onto the red-half and malware findings:

- **Sealed command path.** Open the envelope under the field key, guard the command byte against the andon set, bound the zone to the provisioning band, verify the sequence and the state tag.
- **Anti-replay window.** `last_seq` and a strictly monotonic sequence rule.
- **Authenticated state tag.** A keyed tag over the nine-byte authorization record, computed with a domain-separated nonce and verified in constant time.
- **Fault-clear authorization.** A local request raises the pending indication and never bypasses authorization.
- **No untrusted task execution.** The production firmware never runs data it did not authorize; the check-in and tasking gates stay closed and the bot path is compiled out.
- **Fail safe.** Retract the gate at boot, deploy to divert on link loss, and on every fault, and return the fail-safe zone.
- **Bot removal.** Close the check-in and tasking gates, clear the runtime state and the marker, erase the reserved sector, and remove the re-install code path together, because either one alone is insufficient.
- **Build integrity.** Do not define `SANDBOX_ONLY` in production, and sign and verify the firmware image.
- **Debug lockdown.** On the deployed part, burn secure-boot and debug-disable in OTP so SWD cannot read or write SRAM.

XI.V. What the Hardening Buys, and What It Does Not

The command path closes the forgery, replay, and verdict-tamper surfaces:

- A forged or modified frame fails the tag before parsing.
- A captured valid command fails on freshness on second use.
- A debugger-written verdict fails the state-tag check before the diverter moves.
- The node identity is bound into the associated data, so a frame cannot be relabeled for another node.

It does not, by itself, stop a payload that listens below the authenticated path or that registers with its own listener, and it does not remove the copy of the payload in the reserved sector. That is a build-integrity, authorization- boundary, and state-erasure problem, not a protocol problem, and it is the central lesson of the act.

XII. Implementation Compliance Mapping

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the andon policy; `src/diverter.c` sequences the actuator and fails safe to divert; `src/sensor.c` samples the DHT11 line band; `src/display.c` renders the andon readout; `src/status_led.c` maps the verdict to the red, yellow, and green lamps; `src/button.c` debounces the fault-clear request; `src/servo.c` drives the diverter PWM; `src/ir_remote.c` decodes NEC fault-clear commands.
- **Command and state:** `src/control.c` opens and applies sealed commands with a guarded command set and a bounded zone band; `src/andon_auth.c` holds the authorization record, the monotonic anti-replay window, and the authenticated state tag.
- **Malware:** `src/implant.c` implements the SANDBOX_ONLY C2V1 magic listener, the six-byte check-in, the benign task handler, the reserved-sector bot marker, re-install on boot, and the CoreDebug anti-debug trap.
- **Radio:** `src/radio.c` provisions the transceiver, builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`, and `gen_banner.py`.
- **Classroom:** `scripts/gateway.py` (gateway provisioning, authentication, CSV logging, sealed command replies), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.

- **Tests:** 146 native C cases and 479 checks with 0 failures. They cover the full DHT waveform and every timeout shape, the diverter state machine and its bounded travel, the sealed command path and its guards, the authorization window and state tag, the fault-clear no-bypass path, fail-safe on link loss, the declared-length parser, the servo and LED mappings, the fault-clear remote paths, the bot first run, re-install on boot, the C2V1 magic, the check-in, the benign task handler and the check-in and tasking gates, the bot marker, anti-debug, and the cryptographic primitives against published vectors.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `diverter.c`, `control.c`, `andon_auth.c`, `chacha20.c`, `poly1305.c`, `crypto_aead.c`, `blake2b.c`, `argon2.c`, `crypto_kdf.c`, `envelope.c`, `monitor.c`, and `implant.c` under LLVM source coverage, for 2114 / 2114 lines. `main.c` is excluded from coverage by design.

XIII. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at $t=3$, $p=1$, $m=64$ blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host gateway.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so the firmware and the Python gateway derive the same key in the classroom. Production firmware must provision key material from OTP memory at manufacture and must never embed a passphrase, salt, or derived key in flash.
- **Open debug port.** The Debug Probe is the instrument for both the malware analysis and the verdict-tamper exercise. The authenticated state tag makes a tampered verdict detectable, but a probe that can read the field key from SRAM defeats the design. Production must disable debug in OTP.
- **Replay scope.** The window rejects a replayed command, but a reboot resets `last_seq` to zero. A command captured before a reboot can therefore be

replayed after one. A production controller persists the sequence floor in non-volatile memory.

- **The bot is benign, guarded, and breadboard-bound.** It is compiled only under `SANDBOX_ONLY`, confined to the breadboard, and has no network. It touches only its own radio frames, its runtime flags, and the reserved sector on the same chip. It does not survive a deliberate sector erase and does not resist physical forensics.
- **The C2 is a local script.** The listener is a local classroom script on the student's own radio and network id. There is no external address, no internet path, and no external network. The tasks are benign: a tower light blink, a synthetic log line, and a status report. Any claim of an operational botnet would be an overclaim.
- **The bot listens below the protocol entirely.** No amount of wire authentication stops a module that reads the raw payload before the envelope is opened or that answers its own listener. Mitigating that is a build-integrity, signing, authorization-boundary, and debug-lockdown problem, not a protocol problem.
- **Fail mode trade-off.** The line is fail-safe to the divert posture and the fault-clear request cannot bypass authorization. Any change to either must be a policy decision, not a code accident.
- **Infrared path unauthenticated.** The NEC fault-clear remote has no key and no anti-replay state. Any compatible remote can send a request. The sealed radio path is the authorization path; the optical surface is a documented exposure.
- **Sensor trust boundary.** The DHT11 is a check-summed but not authenticated one-wire sensor; the line band is only as trustworthy as the physical wiring and the edge timing.
- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not availability.

XIII.I. Future Work

- Provision the field key from RP2350 OTP memory and add a documented rotation procedure.

- Persist the anti-replay sequence floor in non-volatile memory so a reboot does not reset freshness.
- Add a signed-image verification step to the flash procedure and a measured boot chain on the RP2350.
- Add a reserved-sector erasure step to the documented flash procedure so the command-and-control lesson maps to a repeatable remediation.
- Add a tasking allowlist and a runtime authorization boundary so no raw frame can reach a task handler.
- Add a check-in rate limit and a fleet-level listener inventory so an unexpected cadence is visible.
- Burn debug-disable and secure-boot settings in OTP for the deployed part.
- Add an actuator-state ledger and a fault-clear de-bounce audit.
- Raise the Argon2id profile on the gateway and record the derivation cost as a measured parameter.
- Extend the malware track toward the Act VIII availability lesson with a controlled lockout channel and its containment procedure.

XIV. Conclusion

OPERATION IRON CHOIR turns a trusting station into a defended one, and then shows why a defended protocol is not the whole story. The node drives the full Embedded Hacking peripheral set, so a state failure has a visible and physical consequence at the line diverter. The LoRa command path is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the andon node identity bound as associated data. Act VII adds a guarded command set and a bounded zone band, a monotonic anti-replay window so a captured command dies on second use, an authenticated state tag so a debugger-written verdict dies before the diverter moves, a fault-clear request that asks for authorization instead of bypassing it, and a fail-safe posture that returns the safe zone. The gateway authenticates before it parses, so the spoofing client that once forged a command now fails at the tag, and the replay that once moved a gate now fails at the window. And then there is the bot: a benign, `SANDBOX_ONLY` FROSTLINE module that checks in with C2V1 and the `0xB7` identifier every four ticks, runs a small set of benign tasks, writes a real marker into a reserved flash sector, and re-installs on every boot, all without ever touching the sealed wire. The firmware, gateway toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline,

and the threat model states exactly which assumptions remain. That combination, a sealed command path next to an honest account of the module that takes orders beneath it, is the lesson Act VII owes the story: the protocol was never the hard part. The chain of command was.

XV. References

- [1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.
- [2] Vishay Semiconductors, “IR Receiver Modules for Remote Control Systems (VS1838B),” Vishay Intertechnology, datasheet 81910, 2018.
- [3] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.
- [4] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.
- [5] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechnotalent/Reverse-Engineering>, 2026.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [7] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.
- [8] A. Costin and J. Zaddach, “A large-scale analysis of the security of embedded firmwares,” *Proceedings of the 23rd USENIX Security Symposium*, 2014, pp. 95-110.
- [9] E. Y. Vasserman and N. Hopper, “Vampire attacks: draining life from wireless ad hoc sensor networks,” *IEEE Transactions on Mobile Computing*, vol. 12, no. 2, 2013, pp. 318-332.
- [10] K. Thomas, “The embedded hacking course and breadboard,” <https://github.com/mytechnotalent/Embedded-Hacking>, 2026.